安恒信息
DAS-security 安全中国



Pytorch 说明

杭州安恒信息技术股份有限公司 陈凯
www.dbappsecurity.com.cn

安恒信息

章节	学习目标
数据处理	■ 数据处理
模型构建	■ 模型构建
模型训练	■ 模型训练
模型评估	■ 模型评估
pytorch优化器	■ pytorch优化器

安恒信息

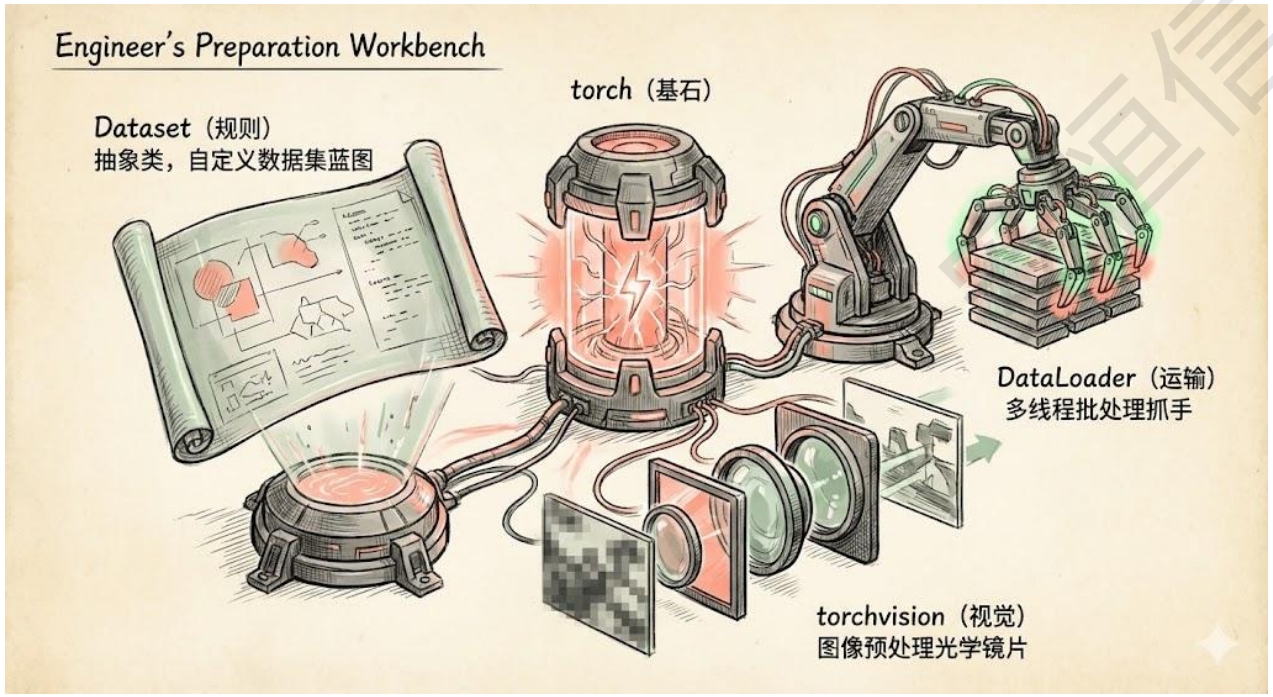
Part 01

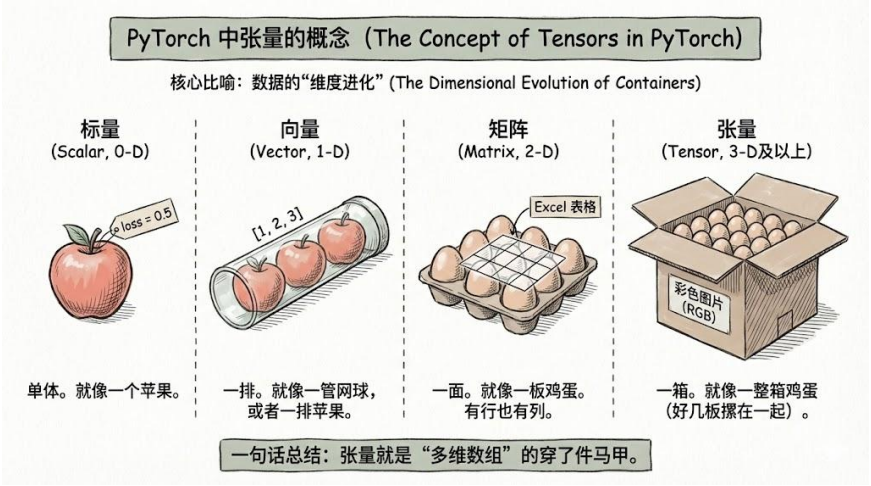
数据处理

数据处理

导入模块：在深度学习中，模型训练数据是关键。PyTorch提供了简洁而强大的工具，用于处理不同类型的数据。核心思想是将**原始数据**处理为模型可以接受的形式（**张量**），并通过**批处理**（Batching）加速训练。

```
import torch
from torch.utils.data import DataLoader, Dataset
from torchvision import datasets, transforms
```





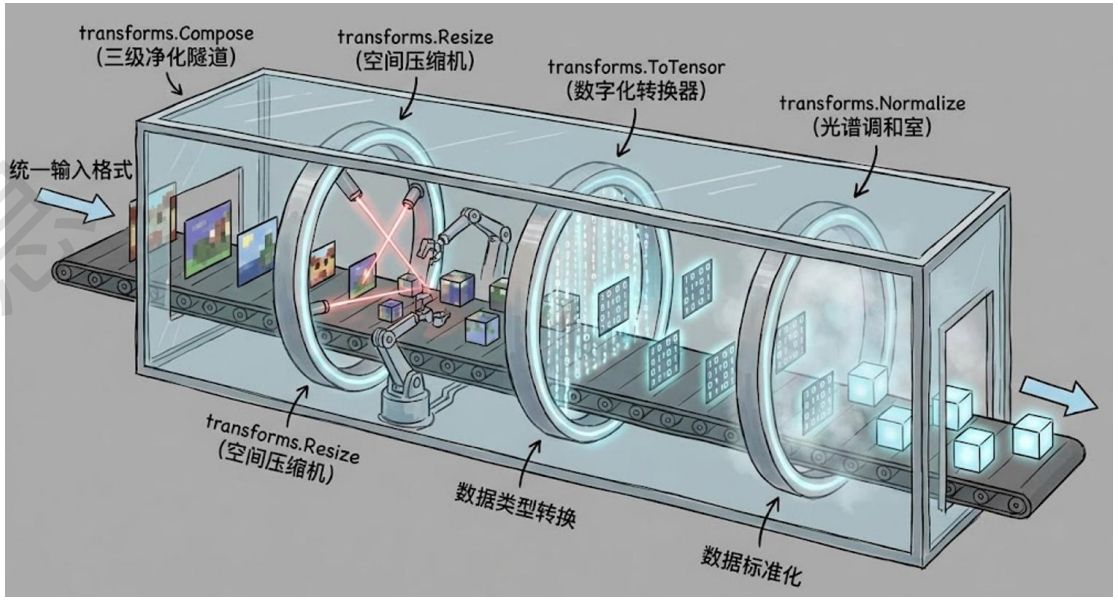
解释：

- **torch**：PyTorch的核心库，包含张量（Tensor）操作、自动微分、模型构建等功能。
- **torch.utils.data.Dataset**：一个抽象类，用于自定义数据集。所有数据集都必须继承这个类。
- **DataLoader**：这个工具可以高效地加载数据，支持多线程读取和批次处理。
- **torchvision**：专门为图像处理准备的工具库，包括常见的数据集和预处理（例如图像增强、转换等）。

数据转换与增强（Data Transformations and Augmentation）

深度学习模型通常要求输入数据格式统一（如图像大小），或者需要对数据进行一些预处理，如标准化、裁剪或旋转。`torchvision.transforms`提供了一些常用的转换函数，尤其是对图像数据。

```
transform = transforms.Compose([
    transforms.Resize((224, 224)),  # 将图像大小调整为 224x224
    transforms.ToTensor(),          # 将图像转换为张量
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
    std=[0.229, 0.224, 0.225])  # 对数据进行标准化
])
```



解释：

- `transforms.Compose()`：将多个转换操作组合在一起，顺序执行。
- `transforms.Resize()`：将图像调整为统一的大小（例如224x224），深度学习模型通常要求输入图像尺寸一致。
- `transforms.ToTensor()`：将图像转换为PyTorch的张量（tensor），以便模型可以处理。
- `transforms.Normalize()`：标准化操作，通过减去均值、除以标准差，使数据符合0均值、单位方差的分布。这有助于模型更快地收敛。原本色彩各异（有的太亮、有太暗）的数据块在这里被统一“调色”，减去均值、除以方差，出来时大家都变成了色调一致、分布均匀的标准模块。

数据转换的意义：

- **统一输入格式**：无论是图像大小还是数据类型，模型的输入通常需要统一的格式和范围。
- **数据增强**：通过数据扩充（如旋转、翻转等），可以增加训练样本的多样性，防止模型过拟合。

安恒信息

加载数据

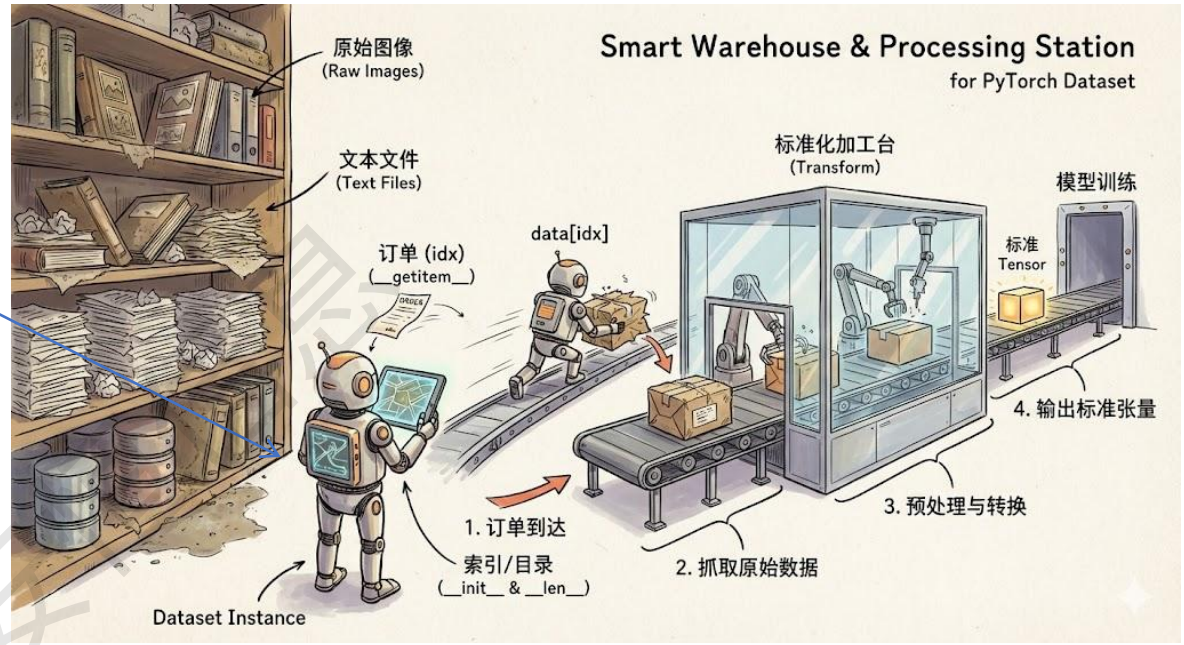
在实际项目中，数据并不总是现成的格式，可能是来自文件、数据库或其他。因此，PyTorch允许你通过继承Dataset类来自定义数据集。

```
class CustomDataset(Dataset):
    def __init__(self, data,
        labels, transform=None):
        self.data = data
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        sample = self.data[idx]
        label = self.labels[idx]
        if self.transform:
            sample = self.transform(sample)
        return sample, label
```

一般写法



解释：

- **__init__()**：构造函数，用于传递数据和标签。`transform`用于在取出数据后进行预处理（例如图像缩放或归一化）。
- **__len__()**：返回数据集的长度，即样本的数量。它用于告诉`DataLoader`如何在批次中划分数据。
- **__getitem__()**：通过索引（`idx`）获取某个样本的数据和标签。在这里，你可以添加数据增强和预处理的逻辑，例如将图像缩放到特定大小，或将文本数据转换为张量。每次`DataLoader`在训练期间加载一批数据时，都会调用该方法来获取数据和标签。

数据划分与加载（Generic Split & Loader）

```
full_dataset = CustomDataset(data=data, labels=labels, transform=transform)
# ---------------------------------------------------------------------------
# 2. 划分训练集与验证集（通用逻辑）
# ---------------------------------------------------------------------------
total_size = len(full_dataset)
train_size = int(0.8 * total_size)        # 80% 用于训练
val_size = total_size - train_size        # 20% 用于验证

# random_split 不关心数据内容，只关心数量，所以它是通用的
train_dataset, val_dataset = random_split(full_dataset, [train_size, val_size])
```



在深度学习中，data可以是多种形式，具体取决于你要处理的数据类型。常见的情况包括：

- **图像数据**：data通常是一个包含多张图片路径的列表，或者是预加载的图像矩阵。
- **文本数据**：data可以是句子、单词列表、嵌入等。
- **数值数据：**data可以是dataframe 或者其他的数值数据。
- **时间序列数据**：data可以是按时间排序的数值或序列。

在具体使用中，data是一个**Python序列对象**（如列表、NumPy数组等），labels是其对应的标签。**每个数据点（sample）与标签（label）一一对应**，它们存储在data和labels的同一索引位置。

安恒信息

数据划分与加载（Generic Split & Loader）

```
# ---------------------------------------------------------------------------
# 3. 创建 DataLoader（通用配置）
# ---------------------------------------------------------------------------
# DataLoader 的作用是将数据打包成 Batch。
# 无论是数字还是路径字符串，它都能把它们打包起来。
train_loader = DataLoader(
    dataset=train_dataset,
    batch_size=4,       # 每次处理 4 条数据
    shuffle=True,       # 训练时打乱顺序，保证模型不记顺序
    num_workers=2       # 适度开启多进程加速（通用建议值）
)

val_loader = DataLoader(
    dataset=val_dataset,
    batch_size=4,
    shuffle=False       # 验证时不打乱，结果可复现
)
```

DataLoader（批量加载器）： 训练流程的“输送带”。它不需要知道数据的内容是图还是表，只负责按照设定的规则（如是否打乱、一次取多少个），源源不断地将数据打包输送给模型。



安恒信息

Part 02

模型构建

模型构建

定义模型结构: 在数据处理部分，我们已经将数据准备好了，包括对数据的预处理（如标准化或归一化）、标签编码以及使用 DataLoader 加载批量数据。在此基础上，接下来需要构建一个神经网络模型，它能够接受我们已经处理好的输入数据，并通过层与层之间的权重学习，来预测结果。

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class SimpleNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(SimpleNN, self).__init__()
        # 定义网络层
        self.fc1 = nn.Linear(input_size, hidden_size)  # 输入层到隐藏层
        self.fc2 = nn.Linear(hidden_size, output_size) # 隐藏层到输出层

    def forward(self, x):
        # 定义前向传播过程
        x = F.relu(self.fc1(x))  # 使用 ReLU 激活函数
        x = self.fc2(x)          # 输出层
        return x
```

我们使用 PyTorch 框架构建模型。PyTorch 中的每个模型都继承自 torch.nn.Module 类，nn.Module 是所有神经网络模块的基类。模型构建的核心包括定义模型的层结构和前向传播逻辑。

安恒信息

定义模型结构

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class SimpleNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(SimpleNN, self).__init__()
        # 定义网络层
        self.fc1 = nn.Linear(input_size, hidden_size)  # 输入层到隐藏层
        self.fc2 = nn.Linear(hidden_size, output_size) # 隐藏层到输出层

    def forward(self, x):
        # 定义前向传播过程
        x = F.relu(self.fc1(x))  # 使用 ReLU 激活函数
        x = self.fc2(x)          # 输出层
        return x
```



1.1 `__init__()`: 模型初始化

这是构建网络层的地方，定义了模型的各个层次。让我们具体讲解每个步骤的作用：

- **`super(SimpleNN, self).__init__()`**:
  这一行确保我们继承了 `torch.nn.Module` 的所有功能。`super()` 用于调用父类的初始化函数，从而初始化 `nn.Module` 的所有基础组件。

- **`self.fc1 = nn.Linear(input_size, hidden_size)`**:
  这行代码定义了第一层的网络结构，即输入层到隐藏层的全连接层（也叫线性层）。`input_size` 是输入数据的特征维度，`hidden_size` 是隐藏层的神经元数量。全连接层的功能是将输入数据与权重矩阵相乘，并加上一个偏置项。

- **`self.fc2 = nn.Linear(hidden_size, output_size)`**:
  这一行定义了隐藏层到输出层的全连接层。`hidden_size` 是前一层的输出大小，`output_size` 是输出类别的数量。通常，分类任务的输出层维度等于类别的数量。

安恒信息

定义模型结构

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class SimpleNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(SimpleNN, self).__init__()
        # 定义网络层
        self.fc1 = nn.Linear(input_size, hidden_size)  # 输入层到隐藏层
        self.fc2 = nn.Linear(hidden_size, output_size) # 隐藏层到输出层

    def forward(self, x):
        # 定义前向传播过程
        x = F.relu(self.fc1(x))  # 使用 ReLU 激活函数
        x = self.fc2(x)          # 输出层
        return x
```

逐步解析前向传播：

- `x = F.relu(self.fc1(x))`：

这里 `self.fc1(x)` 表示输入数据 `x` 通过第一层全连接层（即 `fc1`）的线性变换。这相当于将输入数据 `x` 与一个权重矩阵相乘，并加上偏置。之后通过 `ReLU` 激活函数，增加网络的非线性表达能力。

> **ReLU (Rectified Linear Unit) 激活函数**：`ReLU(x) = max(0, x)`，只保留大于 0 的输出，使得模型能够捕捉非线性关系。如果没有激活函数，神经网络就等同于线性模型，不能解决复杂的非线性问题。

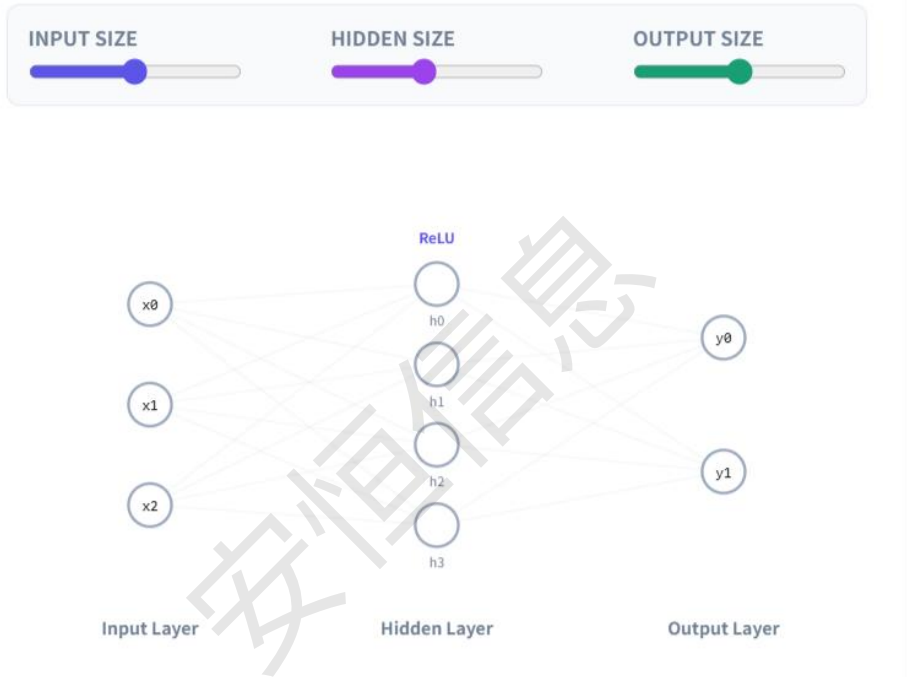
- `x = self.fc2(x)`：

在通过激活函数之后，数据进入第二层，即从隐藏层到输出层的全连接层。这个层的输出不经过激活函数，直接作为模型的最终预测结果。通常，如果是分类任务，下一步会使用交叉熵损失函数来处理这些输出。

模型构建

SIMPLENN.PY

```
class SimpleNN(nn.Module):
  def __init__(self, input_size, hidden_size,
  output_size):
      super(SimpleNN, self).__init__()
  self.fc1 = nn.Linear(3, 4)
  // 输入层 到 隐藏层
  self.fc2 = nn.Linear(4, 2)
  // 隐藏层 到 输出层
```



结构定义 (__init__)

这里我们定义了网络的“骨架”。 试着拖动上方的滑块，你会发现代码中的参数和右侧的网络结构是**实时对应**的。

nn.Linear 本质上就是创建了两层神经元之间的全连接连线（权重矩阵）。

SIMPLENN.PY

```
def forward(self, x):
  # x 输入进来了
  out = self.fc1(x)
  out = F.relu(out)
  out = self.fc2(out)
  return out
```

安恒信息

Part 03

模型训练

模型训练流程

模型训练的核心目标是通过输入数据不断调整模型的参数，以最小化预测结果和真实标签之间的误差。这个过程分为以下几个关键步骤：

- **定义损失函数**：损失函数用于衡量模型预测值和真实标签之间的差异。常用的损失函数包括交叉熵损失（用于分类任务）和均方误差（用于回归任务）。
- **选择优化器**：优化器用于更新模型参数，以最小化损失函数。常见的优化算法有 SGD（随机梯度下降）和 Adam。
- **训练循环**：通过迭代（epochs）将数据输入模型、计算损失、反向传播梯度，进而更新模型的权重。

在训练开始之前，首先需要定义**损失函数**和**优化器**。损失函数用于计算模型预测值和真实值的差异，优化器则决定了如何调整模型的参数来减少损失。

```
# 定义损失函数
criterion = nn.CrossEntropyLoss()  # 用于多分类任务
```

在分类任务中，使用交叉熵损失函数

```
# 定义优化器
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)  # Adam优化器，学习率为0.001
```

- `model.parameters()`：获取模型中所有需要更新的参数。
- `lr=0.001`：设置学习率（学习率决定每次参数更新的步伐大小）。

安恒信息

训练循环

```python
num_epochs = 10  # 定义训练的轮数

for epoch in range(num_epochs):
    model.train()  # 将模型设为训练模式
    running_loss = 0.0

    for i, (inputs, labels) in enumerate(train_loader):  # 迭代训练数据
        # 清空上一次的梯度
        optimizer.zero_grad()

        # 前向传播：计算模型预测
        outputs = model(inputs)

        # 计算损失
        loss = criterion(outputs, labels)

        # 反向传播：计算梯度
        loss.backward()

        # 更新模型参数
        optimizer.step()

        # 累积损失值
        running_loss += loss.item()

    # 打印每个epoch的平均损失
    print(f"Epoch [{epoch+1}/{num_epochs}], Loss: {running_loss/len(train_loader):.4f}")
```

训练循环是训练模型的核心部分，它包括：

1. **前向传播**：将输入数据通过模型，得到预测结果。
2. **计算损失**：通过损失函数，计算模型预测值和真实标签之间的误差。
3. **反向传播**：根据损失值，计算梯度，更新模型参数以减少误差。

1. `model.train()`：将模型设为训练模式。PyTorch模型有两种模式：`train` 和 `eval`，其中 `train` 模式会启用如 dropout 和 batch normalization 等影响训练的操作，而 `eval` 模式会关闭这些操作。

2. `optimizer.zero_grad()`：在反向传播之前，需要清空之前的梯度值。因为 PyTorch 中的梯度是累积的，所以每次计算完梯度后必须清空之前的值。

3. **前向传播**：将输入数据 `inputs` 通过模型，得到预测结果 `outputs`。通过调用模型的 `forward` 函数，数据会自动流经网络的每一层。

4. **计算损失**：将模型的预测结果 `outputs` 和真实标签 `labels` 输入到损失函数 `criterion` 中，计算模型预测的误差。

5. **反向传播**：调用 `loss.backward()` 计算损失函数相对于模型参数的梯度值。这个过程会使用链式法则，逐层计算每个参数的梯度。

6. **更新参数**：使用 `optimizer.step()` 来根据计算出的梯度更新模型的参数，使得模型朝着最小化损失的方向调整。

7. **打印损失**：在每个 epoch 完成后，打印平均损失值，作为模型训练过程中的反馈。

安恒信息

模型保存和加载

在训练结束后，你可能希望保存训练好的模型，以便将来使用或继续训练。通常我们只保存模型的参数，而不是整个模型结构。

```
# 保存模型的参数
torch.save(model.state_dict(), 'model.pth')  # 保存模型参数为 model.pth 文件
```

```
# 加载模型参数
model = SimpleNN(input_size, hidden_size, output_size)  # 重新定义模型结构
model.load_state_dict(torch.load('model.pth'))  # 加载保存的参数
model.eval()  # 设置为评估模式
```

安恒信息

Part 04

模型预测

模型评估

在机器学习和深度学习中，仅仅依赖训练集的性能并不能准确反映模型的实际效果。评估模型的目的是：

- 检查模型是否存在过拟合现象（在训练集上表现好但在新数据上表现差）。
- 确定模型在真实场景下的可用性。
- 通过评估指标了解模型的优缺点，从而为后续的调优和改进提供依据。

模型评估的基本流程如下：

① 准备评估数据：确保使用的是未见过的数据集（验证集或测试集）。

② 设置模型为评估模式：通过 `model.eval()` 切换模型状态，关闭训练时特有的功能（如 `dropout` 和 `batch normalization`）。

③ 关闭梯度计算：使用 `torch.no_grad()`，以节省内存和计算资源。

④ 前向传播：将评估数据输入模型，得到预测结果。

⑤ 计算评估指标：使用适当的评估指标（如准确率、`F1-score`、均方误差等）评估模型表现。

安恒信息

模型评估

当训练、评估之后，对于得到的最终模型，我们会把它用在具体的场景中进行数据分析和预测，预测代码和评估相似，如下：

```
# 模型预测
model.eval()
predictions = []

with torch.no_grad():
    for inputs in val_loader:  # 遍历测试数据
        outputs = model(inputs)
        _, predicted = torch.max(outputs.data, 1)
        predictions.extend(predicted.tolist())

print(f"Predicted classes: {predictions}")
```

需要注意的是：模型再进行预测的过程中，对于预测数据，需要按照当初训练模型时的处理流程进行处理。

安恒信息

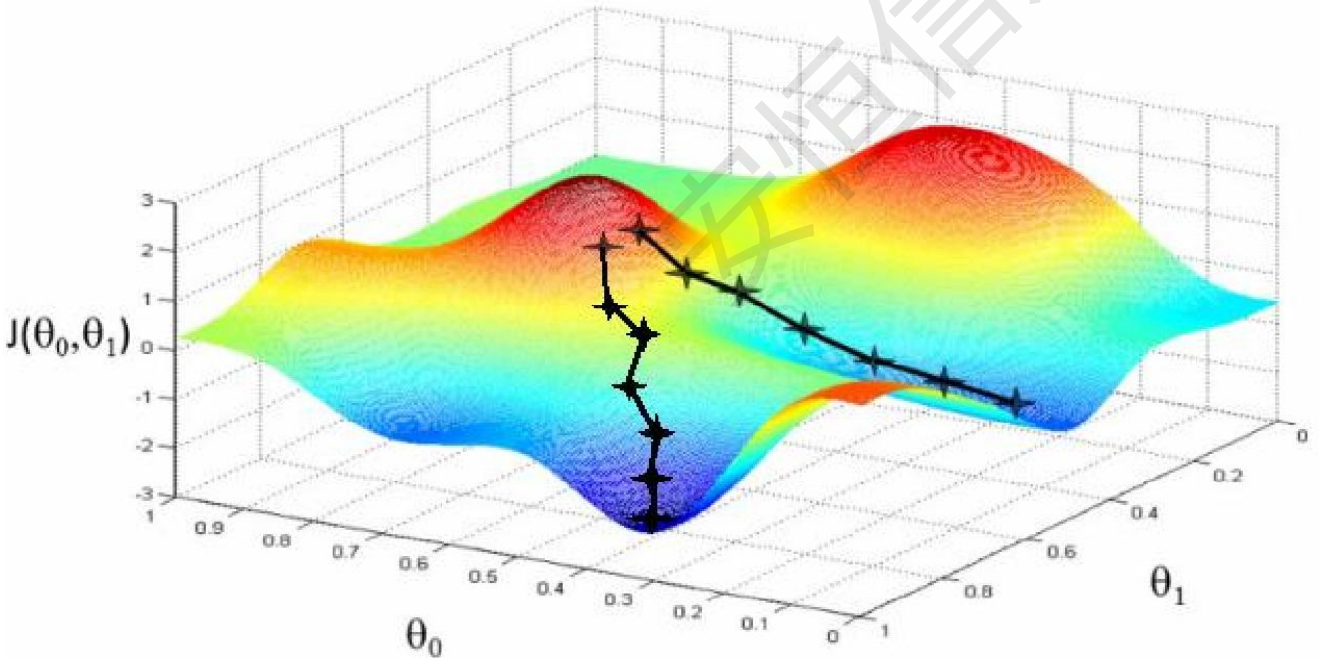
Part 05

Pytorch 优化器

安恒信息

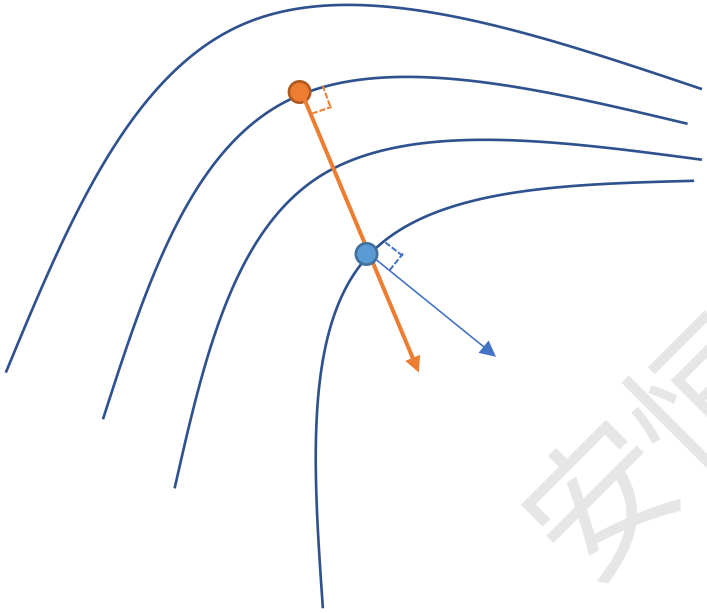
梯度下降

梯度下降算法是一个一阶最优化算法，通常称为最速下降法，是通过函数当前点所对应梯度（或者近似梯度）的反方向，使用规定步长距离进行迭代搜索，从而找到一个函数的局部最小值，最好的情况是找到全局最小值。



安恒信息

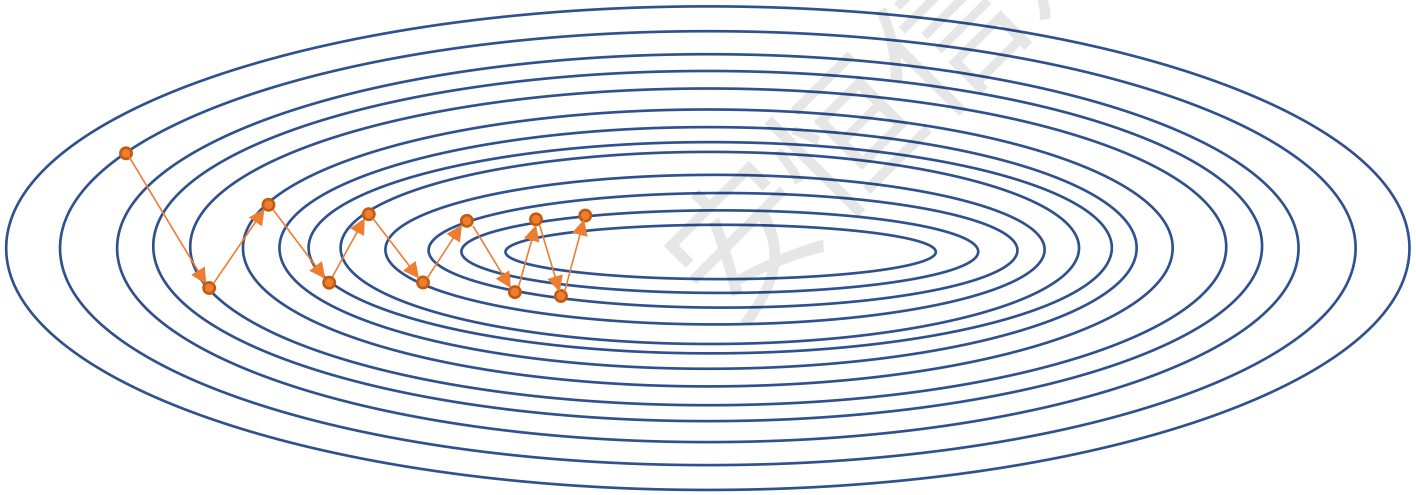
一、优化下降路径：当步长固定，那么此时的下降路径就可能不是最优的：



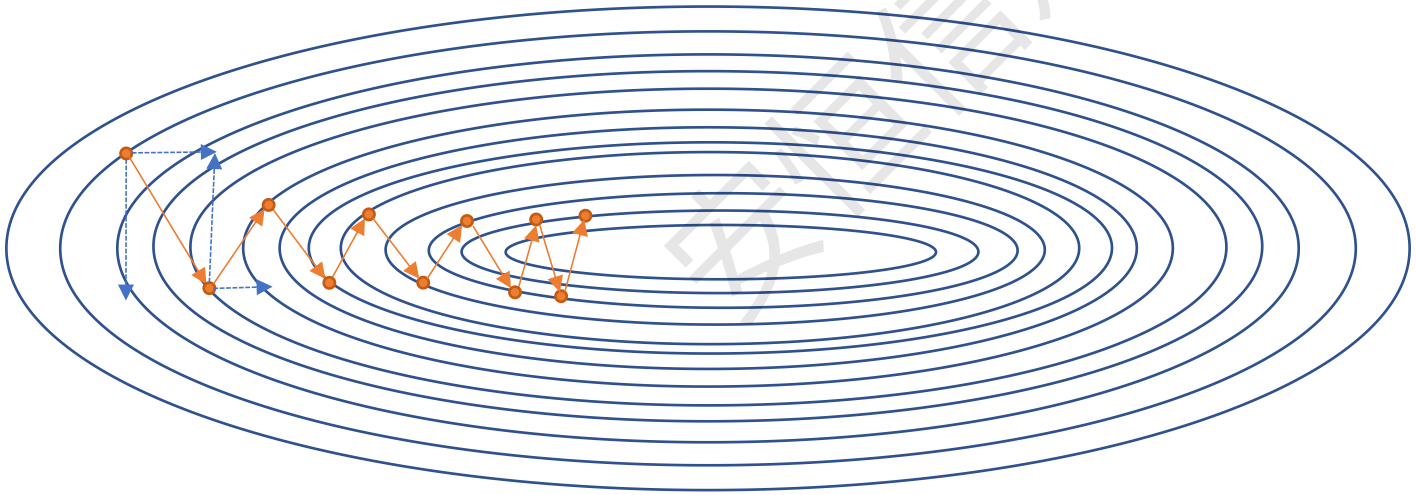


二、优化速度：优化方向确定，加快向极值点的移动速度。

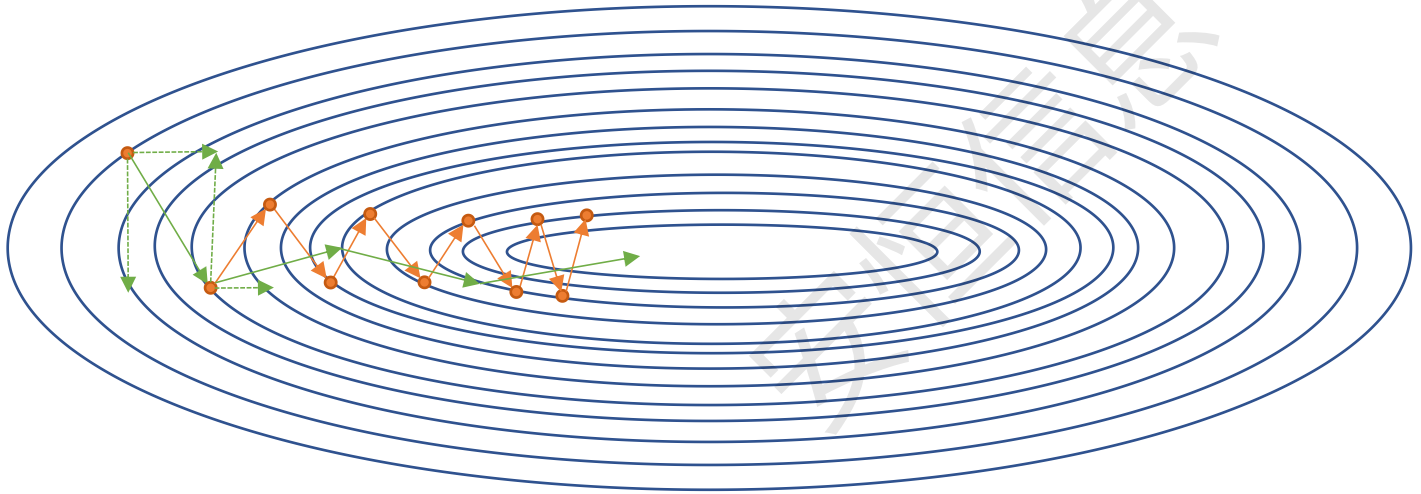
SGD 的更新过程：



SGD 的更新过程：



SGD-Momentum 的更新过程：



SGD 更新公式：

$$\Delta w_{(t)i} = \frac{\partial J(w_{(t-1)i})}{\partial w_i}$$

$$w_{(t)i} = w_{(t-1)i} - \eta \Delta w_{(t)i}$$

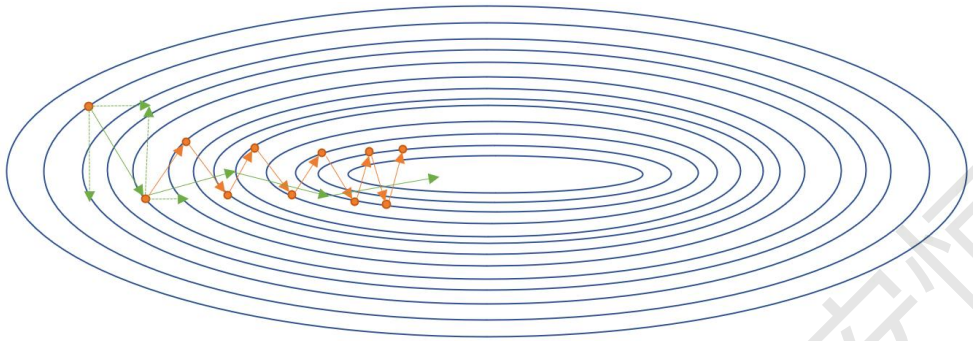
SGD-Momentum 的更新公式：

$$\Delta w_{(t)i} = \frac{\partial J(w_{(t-1)i})}{\partial w_i}$$

$$V_{(t)} = V_{(t-1)} + \Delta w_{(t)i}$$

$$w_{(t)i} = w_{(t-1)i} - \eta V_t$$

SGD-Momentum 的更新过程：



对历史数据进行指数加权平均：

令

$$V_{(t)} = \beta \cdot V_{(t-1)} + (1-\beta) \cdot \Delta w_{(t)i}$$

$$w_{(t)i} = w_{(t-1)i} - \eta V_t$$

可以看到：更新后，越近的历史数据权重更大，越远的历史数据权重越小

$$V_{(t)} = \beta \cdot V_{(t-1)} + (1-\beta) \cdot \Delta w_{(t)i}$$

$$V_{(t)} = 0.9 \cdot V_{(t-1)} + (1-0.9) \cdot \Delta w_{(t)i}$$

$$V_{(t)} = 0.1 \cdot 0.9^0 \cdot \Delta w_{(t)i} + 0.1 \cdot 0.9^1 \cdot \Delta w_{(t-1)i} + 0.1 \cdot 0.9^2 \cdot \Delta w_{(t-2)i} + \ldots + 0.1 \cdot 0.9^{t-1} \cdot w_{(1)i}$$

安恒信息

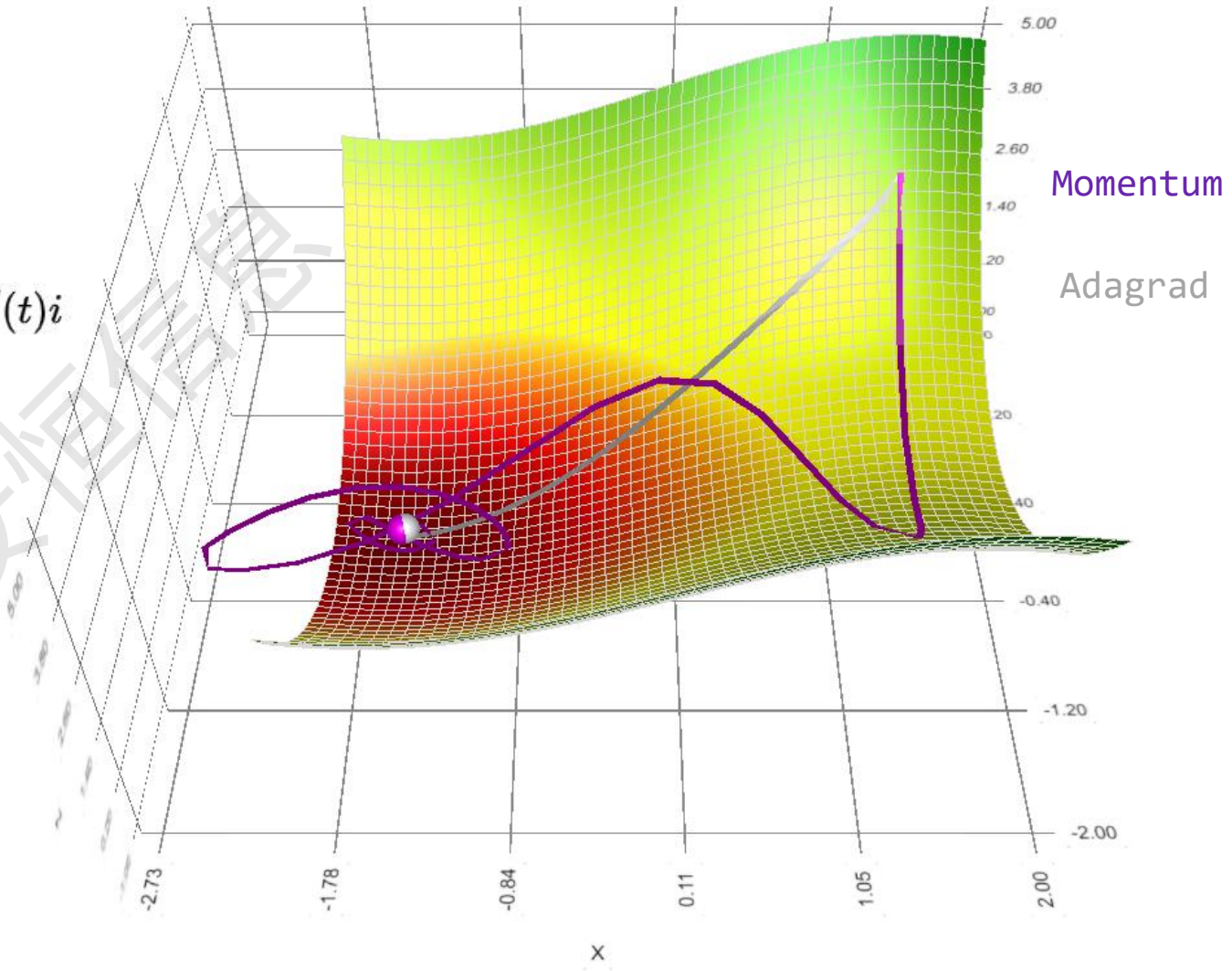
Adagrad 方法

Adagrad 算法通过修改学习率自适应梯度：

$$w_{(t)i} = w_{(t-1)i} - \frac{\eta}{\sqrt{S_{(t)}} + \epsilon} \cdot \Delta w_{(t)i}$$

$$S_{(t)} = S_{(t-1)} + \Delta w_{(t)i} \cdot \Delta w_{(t)i}$$

$$\Delta w_{(t)i} = \frac{\partial J(w_{(t-1)i})}{\partial w_i}$$



安恒信息

对 AdaGrad 进行指数加权的优化，我们可以得到 RMSprop 方法：

$$w_{(t)i} = w_{(t-1)i} - \frac{\eta}{\sqrt{S_{(t)}} + \epsilon} \cdot \Delta w_{(t)i}$$

$$S_{(t)} = \beta S_{(t-1)} + (1-\beta)\Delta w_{(t)i} \cdot \Delta w_{(t)i}$$

$$\Delta w_{(t)i} = \frac{\partial J(w_{(t-1)i})}{\partial w_i}$$

安恒信息

Adam = RMSprop + 动量法

$$V_{(t)} = \beta \cdot V_{(t-1)} + (1-\beta) \cdot \Delta w_{(t)i}$$

$$S_{(t)} = \alpha \cdot S_{(t-1)} + (1-\alpha) \cdot \Delta w_{(t)i} \cdot \Delta w_{(t)i}$$

$$w_{(t)i} = w_{(t-1)i} - \frac{\eta}{\sqrt{S_{(t)} + \epsilon}} \cdot V_{(t)}$$

安恒信息

Pytorch 优化器

类	算法名称
torch.optim.SGD()	随机梯度下降算法
torch.optim.Adagrad()	Adagrad 算法
torch.optim.Adam()	Adam 算法
torch.optim.RMSprop()	RMSprop 算法
torch.optim.Adadelta()	Adadelta 算法
torch.optim.LBFGS()	L-BFGS 算法
torch.optim.Adamax()	Adamax 算法
torch.optim.Rprop()	弹性反向传播算法
torch.optim.ASGD()	平均随机梯度下降算法

安恒信息

安恒信息
DAS-security 安全中国

谢谢观看

Thanks for watching



安恒信息

杭州安恒信息技术股份有限公司 XXX

www.dbappsecurity.com.cn